FAST — NATIONAL UNIVERSITY OF COMPUTER AND EMERGING SCIENCES
FAST School of Computing · Karachi Campus · Department of Artificial Intelligence & Data Science
AL-2002 Artificial Intelligence (Laboratory) · Fall 2026

PROJECT PROPOSAL

Semester Lab Project · Weightage 1 mark

Group Information

	Name	Roll Number	Section	Role in the project
Leader	Muhammad Daniyal Alam	24K-2048	BCY-5A	Sole Developer: Full implementation of the project

Project Title	2048 AI with Expectimax
Project Idea	Idea No. 6 from the Project Ideas list
Category	Part A — Game ☑ Part B — Applied / Data ☐
GitHub Repository URL	https://github.com/mdaniyalalam/2048-Expectimax-AI
Date of Submission	October 20, 2026

1. Project Title and Summary

Title: 2048 AI with Expectimax

Summary: This project is a fully playable version of the popular sliding tile puzzle game 2048, featuring both a manual player mode and an automated artificial intelligence solver. It is designed for puzzle enthusiasts who want to play the classic game, as well as students and developers interested in observing how an AI tackles decision-making under uncertainty. In 2048, new tiles spawn randomly in empty spaces after every move, making it impossible to plan a guaranteed sequence of steps in advance. To overcome this, the software uses an Expectimax search algorithm that "thinks ahead" by calculating the mathematical average of all possible random tile spawns before deciding on the best move. The AI's foresight is guided by custom evaluation rules that score the board based on human-like strategies, such as keeping the largest tiles tucked safely in a corner and maximizing empty space. Ultimately, this allows the AI to navigate the game's randomness and consistently reach high-value tiles, achieving scores that a plain, rule-based program could not accomplish.

2. Problem Statement and Motivation

The project addresses the challenge of sequential decision-making in highly stochastic environments, specifically modeled through the puzzle game 2048. 2048 is deceptively difficult to solve by hand or brute force due to its immense search space and inherent uncertainty. After each player move, the environment randomly spawns a '2' or '4' tile in any empty cell. Because the branching factor depends on the number of empty tiles and two possible tile values, a single move can generate up to 30 immediate environmental responses. Looking ahead just three turns expands the search tree to hundreds of thousands of probabilistic states, rendering blind enumeration and standard deterministic search completely useless. AI developers and puzzle enthusiasts face this challenge constantly; humans cope by intuitively applying positional heuristics, such as anchoring the largest tile in a corner, rather than calculating exact probabilities. Solving this problem computationally using Expectimax is worthwhile because it demonstrates how an intelligent agent can optimize long-term utility in unpredictable real-world scenarios where environments act randomly rather than adversarially.

3. Problem Formulation

3.1 PEAS Description

Component	Description for your project
Performance measure	The primary metric is the highest tile reached on the board (e.g., 2048, 4096) before a game-over state. Secondary measures include the cumulative game score (gained by merging tiles) and the total number of moves survived
Environment	A 4 × 4 spatial grid where cells are either empty or contain a tile with a value that is a power of 2 (2, 4, 8, …). After every valid move, the environment randomly spawns a new tile (90% chance of a '2', 10% chance of a '4') in an available empty space
Actuators	The agent outputs one of four discrete directional commands: Up, Down, Left, or Right. These commands slide all tiles in the chosen direction and merge adjacent identical tiles.
Sensors	The agent perfectly reads the current state of the 4 × 4 matrix, identifying the exact values and positional coordinates of all tiles and empty spaces before making a decision.

3.2 Environment and Agent Type

Property	Classification — tick one
Observability	Fully observable ☑ Partially observable ☐
Determinism	Deterministic ☐ Stochastic ☑
Change over time	Static ☑ Dynamic ☐
State and actions	Discrete ☑ Continuous ☐
Agents	Single-agent ☑ Multi-agent / adversarial ☐
Agent type	Simple reflex ☐ Model-based ☐ Goal-based ☐ Utility-based ☑ Learning ☐

3.3 Problem Representation

Element	Your representation
State representation	A 4 × 4 integer matrix where each cell contains either empty (0) or a power of 2 representing a tile's current value.
Actions / moves	Four discrete directional moves: Up, Down, Left, and Right.
Initial state	A 4 × 4 matrix initialized with mostly empty cells and exactly two randomly placed tiles (each carrying a 90% probability of being a '2' and a 10% probability of being a '4').
Goal or terminal test	The terminal test is triggered when the board is in a "Game Over" state, meaning there are zero empty cells left and no adjacent tiles horizontally or vertically share the same value to allow a merge.
Cost, utility, fitness or evaluation	A custom utility function evaluated at the leaf nodes of the search tree. It calculates a weighted sum based on four heuristics: anchoring the maximum tile in a corner, maximizing empty cell count, maintaining grid monotonicity, and penalizing adjacent tile variance (smoothness).

4. Proposed AI Approach

#	Technique	Role in the project	Own code or library
1	Expectimax Search	Explores a limited-depth game tree alternating between player actions (max nodes) and random tile spawns (chance nodes) to find the optimal move.	Own code
2	Heuristic Evaluation	Assigns a utility score to future board states based on human-like 2048 strategies, guiding the Expectimax search.	Own code
B	Baseline: Random Play	Serves as a control variable by picking a random valid direction, measuring how much Expectimax improves performance.	Own code
B	Baseline: Greedy Search (Depth-1)	Acts as a secondary baseline that maximizes immediate heuristic score without anticipating future tile spawns.	Own code

Expectimax search is perfectly suited for 2048 because the environment is stochastic rather than strictly adversarial. Instead of assuming an opponent makes the worst move, Expectimax calculates the mathematical average of all possible random tile spawns, optimizing for the highest probable utility. Because the search space expands rapidly, tree depth is limited to 2-3 moves. To assess these future states, my evaluation function scores the board using four adjustable heuristics: anchoring the largest tile in a corner, maximizing empty tiles, promoting monotonicity, and penalizing adjacent tile variance. To rigorously test the system, I will compare Expectimax against two alternatives. A baseline Random Agent will randomly pick valid moves to measure basic competence, while a Greedy Agent (Depth-1 search) will show whether multi-step Expectimax planning actually outperforms short-term heuristic maximization.

4.1 Planned Class Design

1. Board2048 (The Environment):
Attributes:
- grid, score

Methods:
- move(direction), get_valid_moves(), spawn_tile(), is_game_over(), copy()

Relationship: An independent class representing the game state. It is passed as an argument to the agents so they can simulate future moves.

2. BaseAgent (Abstract Parent Class):
Attributes: None required strictly for logic
Methods:
- select_move(board)

Relationship: The base class establishing polymorphism for the game loop, ensuring any agent can be swapped in seamlessly.

3. RandomAgent (Subclass):
Attributes: Inherits from BaseAgent
Methods:
- select_move(board)

Relationship: Child of BaseAgent; serves as the primary experimental baseline.

4. ExpectimaxAgent (Subclass):
Attributes:
- depth, nodes_expanded, w_corner, w_empty, w_mono, w_smooth

Methods:
- select_move(board), expectimax(board, depth, is_player_turn), evaluate(board)

Relationship: Child of BaseAgent. It houses the core artificial intelligence logic for the project.

5. Scope

In scope:

#	Feature	Why it is essential
1	Playable 2048 Game Engine	Provides the foundational environment, grid logic, and Pygame graphical interface required for both human play and AI integration.
2	Expectimax Search Agent	The core algorithmic requirement for this project, necessary to navigate the game's stochastic tile-spawning mechanics at a search depth of 2 to 3.
3	Adjustable Heuristic Evaluation	Essential for scoring the leaf nodes of the Expectimax tree; relies on four weighted rules (corner anchoring, empty tiles, smoothness, monotonicity) to guide decision-making.
4	Auto-Play Toggle Mode	Allows the Expectimax agent to execute moves automatically without human intervention, which is necessary for running high-speed simulations.
5	50-Game Benchmark Evaluation	Required to statistically prove the Expectimax agent's superiority by comparing its highest tile reached and average score against a Random/Greedy baseline.

Out of scope:

The Expectimax search depth is strictly capped at 3; depths of 4 or higher are out of scope because the massive branching factor slows down real-time move calculation to an unplayable degree. Deployment to web or mobile platforms is excluded; the project will run entirely as a local desktop application using Python and Pygame.

6. Data or Environment Source

Source name and URL	Self-contained Python Environment
Type	Public dataset ☐ Web API ☐ Self-collected ☐ Generated / simulated ☑
Access requirements	No key needed ☑ Free key required ☐ (Paid sources are not permitted)
Approximate size	Negligible (in-memory 4×4 integer matrices).
Licence / terms of use	N/A (Original student code)
Offline fallback	The game runs 100% offline without network requests.

7. Evaluation Plan

What will be compared	The Expectimax Agent (at search depths 2 and 3) will be compared against a Greedy Agent (Depth 1) and a Random Agent.
Metric(s)	Highest tile reached, average total game score, and average nodes expanded per move.
Baseline	Random Agent (selects random valid moves) and Greedy Agent (selects the best immediate move without evaluating chance nodes).

Test instances / test set	Full 2048 game simulations running from the initial empty state until a terminal "Game Over" state is reached.
Number of runs and seed	50 complete game runs per agent configuration, utilizing a fixed seed (random.seed(42)) to ensure exact reproducibility of the tile spawns.
Planned charts (at least three)	Bar chart, Box plot, Line chart

8. Division of Work

Member	Modules / components owned	Approximate share
Muhammad Daniyal Alam	Full project lifecycle	100% (Solo Project)

9. Milestones and Timeline

#	Milestone — what will be demonstrably working	Target date	Owner
1	Playable 2048 game engine in Pygame with manual arrow-key controls, accurate tile merging, and randomized tile spawning.	Oct 28, 2026	Muhammad Daniyal Alam
2	Random and Depth-1 Greedy agents successfully complete auto-play games; evaluation function calculates board utility using the 4 core heuristics.	Nov 04, 2026	Muhammad Daniyal Alam
3	Expectimax agent successfully plays the game at Depth 2 and Depth 3.	Nov 11, 2026	Muhammad Daniyal Alam
4	Headless experiment script successfully runs 50 seeded games for all three agents and generates the 3 required comparative Matplotlib charts.	Nov 18, 2026	Muhammad Daniyal Alam
5	Experiments complete, final integration, documentation and repository cleanup.	Nov 23, 2026	Muhammad Daniyal Alam

10. Expected Outcome and Risks

A working submission will feature a Pygame interface where users can manually play 2048 or toggle an auto-play mode to watch the Expectimax agent solve the board. I expect the final evaluation to show Expectimax consistently reaching the 1024 or 2048 tile, vastly outperforming the Random and Greedy baselines in average score. The primary risk is the massive branching factor making Depth 3 search too slow for real-time play. If this occurs, I will cap chance node evaluations to only the top empty cells or restrict the depth to 2. A secondary risk is heuristic weight conflicts causing board lock-ups. I will mitigate this by running headless batch simulations to systematically tune the weights for optimal performance.

For office use — Lab Instructor	
Proposal status	Approved ☐ Approved with changes ☐ Revision required ☐
Comments	
Signature and date	

End of Project Proposal